

⚡ Clase 12 – Programación Dinámica en una Dimensión

Contar cambio con repetición y sin repetición

En esta clase aprenderemos a resolver dos variantes clásicas del problema de conteo de formas para dar cambio usando Programación Dinámica de una dimensión.

🔄 Recordatorio de la clase anterior

🧠 ¿Qué es la Programación Dinámica (DP)?

La **Programación Dinámica** es una técnica que nos permite resolver problemas complejos dividiéndolos en **subproblemas más pequeños** y almacenando sus resultados para **no repetir cálculos innecesarios**.

Es especialmente útil cuando el problema tiene:

- **Subestructura óptima**: la solución del problema grande se construye a partir de soluciones más pequeñas.
- **Subproblemas superpuestos**: los mismos subproblemas aparecen una y otra vez.

📁 Las dos formas de aplicar DP

1 Top-Down (Memoización)

- Empezamos resolviendo el problema grande.
- Llamamos recursivamente a los subproblemas.
- Cada vez que resolvemos un subproblema, **guardamos su resultado** en una tabla.
- Si se vuelve a necesitar, se reutiliza sin recalcularlo.

👉 Es como resolver recursivamente, pero “recordando” los resultados.

2 Bottom-Up (Tabulación)

- Construimos una tabla `dp[]` empezando desde los **casos base**.
- Vamos llenando la tabla de menor a mayor hasta llegar a la respuesta final.
- No usamos recursión.

👉 Es como construir la solución desde abajo, paso a paso.

2. Problema: Contar cambio **con repetición**

📖 Enunciado

Dado un conjunto de monedas `coins = {c1, c2, ..., cn}` y una cantidad objetivo `target`, contar cuántas formas existen de formar `target` **pudiendo usar una moneda las veces que quieras**.

🧠 Idea clave

Se trata de un problema de **combinaciones**, no de permutaciones.

La DP se recorre:

- monedas en el ciclo externo
- cantidades en el ciclo interno (de menor a mayor)

Ecuación de transición

```
dp[x] += dp[x - coin]    si x >= coin
```

Inicialización

```
dp[0] = 1
```

Ejemplo de código con repetición

```
def count_change_repetition(coins, target):  
    dp = [0] * (target + 1)  
    dp[0] = 1  
  
    for coin in coins:  
        for x in range(coin, target + 1):  
            dp[x] += dp[x - coin]  
  
    return dp[target]
```

3. Problema: Contar cambio **sin repetición**

Enunciado

Dado un conjunto de monedas **coins** y una cantidad **target**, contar cuántas formas existen de formar **target** **usando cada moneda como máximo una vez**.

Idea clave

La diferencia está en el orden:

Para evitar usar varias veces la misma moneda, la DP para las cantidades debe ir **de mayor a menor**.

Diferencia con repetición

El recorrido de **x** cambia:

- Sin repetición → recorrer **x** de **target a coin**
- Con repetición → recorrer **x** de **coin a target**

Ejemplo de código sin repetición

```
def count_change_no_repetition(coins, target):  
    dp = [0] * (target + 1)  
    dp[0] = 1  
  
    for coin in coins:  
        for x in range(target, coin - 1, -1):  
            dp[x] += dp[x - coin]  
  
    return dp[target]
```

4. Comparación visual

Con repetición

```
for coin in coins:  
    for x in coin → target:  
        usar coin varias veces
```

Sin repetición

```
for coin in coins:  
    for x in target → coin:  
        moneda usada solo una vez
```

Problemas: Clase 12 – DP en una dimensión